Software Engineering (UE24CS341A)

Lab-1

Name: Atiksh Gour

SRN: PES1UG24CS096

Class: 5B

Problem Statement: Feature Flag & Dynamic Config Manager

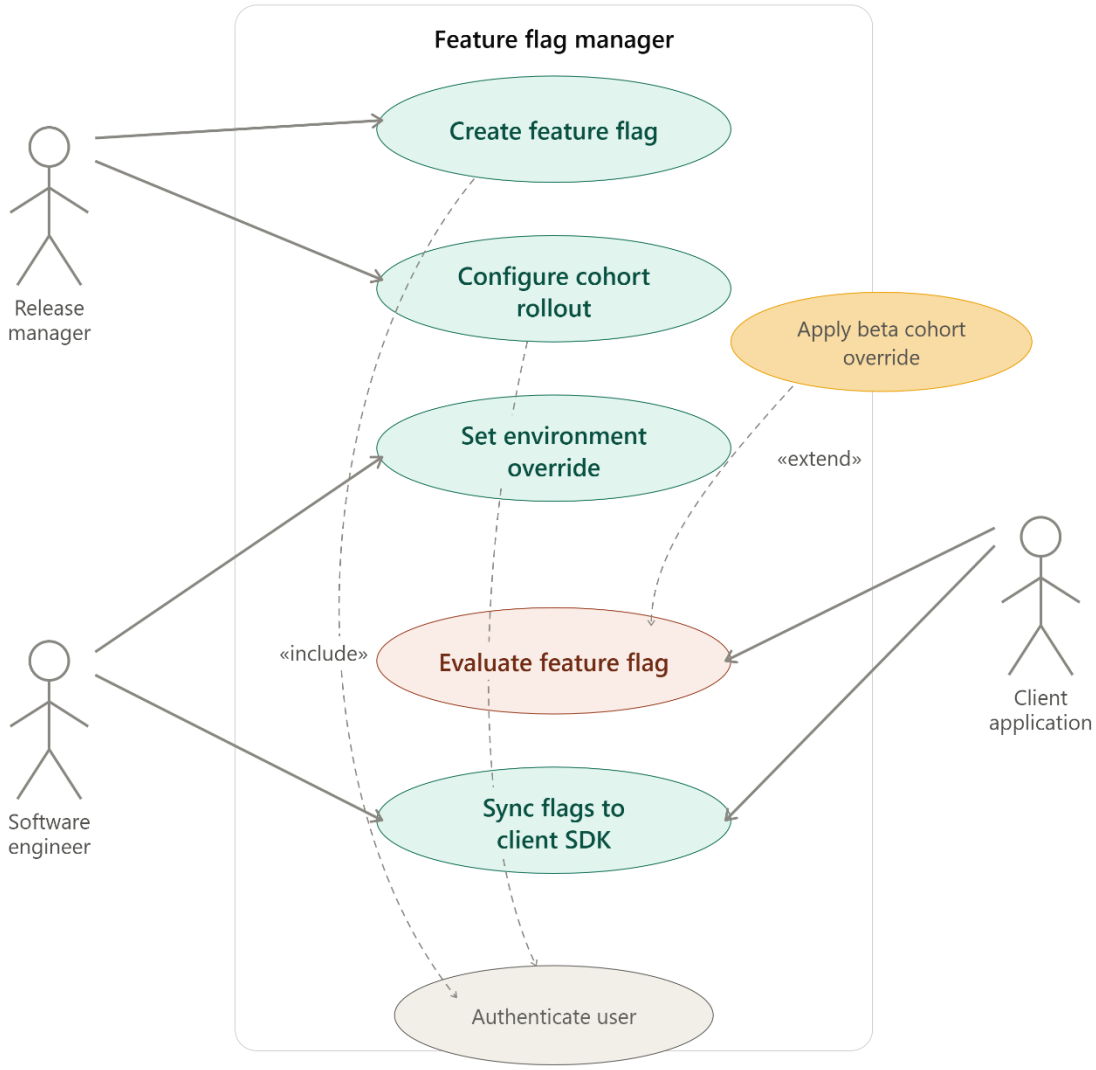
Functional and Non-Functional Requirements:

ReqID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall evaluate user targeting rules (e.g., User ID hash, Beta cohort) and return accurate boolean feature flag states.	High	Pass: Target 10% rollout cohort receives enabled flag; Fail: User outside cohort receives enabled flag.	Core capability of the platform, correct flag evaluation is the primary value delivered to client apps
FR-002	Functional	The system shall allow a Release Manager to create a new feature flag with a unique key, data type (boolean/string/number), and default value.	High	Pass: Submitting a flag creation form with a unique key returns a 201 response and the flag appears in the dashboard within 2 seconds; Fail: Duplicate key is rejected with a	Flags must exist and be uniquely identifiable before any targeting or rollout logic can be applied.

				validation error.	
FR-003	Functional	The system shall allow a Release Manager to set a flag's percentage-based rollout (e.g., 25% of users) using a consistent hashing strategy.	High	Pass: Setting rollout to 25% results in 23–27% of a 10,000-user sample receiving the enabled flag across repeated evaluations; Fail: Same user gets inconsistent results across two calls.	Enables gradual, low-risk feature releases instead of all-or-nothing launches.
FR-004	Functional	The system shall allow a Software Engineer to set environment-specific overrides (Dev, Staging, Production) for a flag's value.	Medium	Pass: Setting a flag to "true" in Staging while "false" in Production returns environment-correct values for each SDK call; Fail: A Staging override leaks into Production evaluation	Supports safe testing in lower environments without affecting live users.
FR-005	Functional	The system shall push updated flag states to all connected client SDKs in real time without requiring a client restart.	High	Pass: A flag toggle is reflected in a connected SDK client within 5 seconds,	Pass: A flag toggle is reflected in a connected SDK client within 5 seconds,

				verified via timestamp logs; Fail: Client shows stale flag value after 5 seconds.	verified via timestamp logs; Fail: Client shows stale flag value after 5 seconds.
NFR-001	Non-Functional	The feature flag evaluation API must execute in under 15 ms to minimize overhead when integrated into client critical paths.	High	Pass: Benchmarking tests confirm target latency and security standards under simulated peak load.	Flag checks sit on the critical path of client apps, so slow evaluation would degrade the host application's performance.
NFR-002	Non-Functional	The system shall restrict flag modification actions in the Production environment to authenticated users with a Release Manager role.	High	Pass: A request to modify a Production flag from a non–Release Manager account returns a 403 Forbidden; Fail: The modification succeeds without proper authorization.	Prevents unauthorized or accidental changes to live production features, reducing risk of outages.

UML Use-Case Diagram:



Use-Case Flow Specification

Use Case: Evaluate Feature Flag

Primary Actor: Client Application (via SDK)

Preconditions:

- The feature flag being requested has already been created in the system (see UC: Create Feature Flag).

- The client application has a valid API key/authentication token.
- A user/session ID is available to evaluate targeting rules against.

Postconditions:

- A boolean (or typed) flag value is returned to the client application.
- The evaluation event is logged for audit purposes.

Main Success Scenario:

1. Client Application sends a flag evaluation request containing the flag key and a user ID to the system's evaluation API.
2. System authenticates the request using the provided API key.
3. System retrieves the flag's configuration, including its targeting rules and default value.
4. System computes a deterministic hash of the user ID to determine cohort placement.
5. System checks whether the hashed value falls within the flag's configured rollout percentage (e.g., 10%).
6. System checks any environment-specific override for the requesting environment (Dev/Staging/Prod).
7. System returns the resolved boolean flag state to the Client Application within the 15 ms performance target.
8. System logs the evaluation event (flag key, user ID, result, timestamp).

9. Use case ends successfully.

Alternate Flow:

6a. Beta Cohort Override Applies (extend)

- 6a1. System detects the user ID belongs to a designated Beta cohort list.
- 6a2. System overrides the standard rollout-percentage result and applies the Beta-specific flag value instead.
- 6a3. Flow resumes at step 7 with the Beta-adjusted value.

5a. User Outside Rollout Cohort

- 5a1. System determines the user's hash falls outside the configured rollout percentage.
- 5a2. System returns the flag's default (disabled) value.
- 5a3. Flow resumes at step 7.